

ProjectDAWIY : 自分好みの DAW 環境を DIY する 共通プラットフォームの構築

山崎 承太郎^{1,a)} 橋田 光代^{1,b)}

概要：ユーザが自身の制作スタイルに合わせて必要な機能を自ら取捨選択し、構築できる「DIY (Do It Yourself)」の思想を取り入れた新しい DAW のコンセプトとして「DAWIY」を提案する。DAWIY の目的は、柔軟な GUI、軽量の動作、そして何よりも高い拡張性を備えたクロスプラットフォームな音楽制作環境の実現である。この要件を満たすため、本研究では Web 技術 (HTML, JavaScript, TypeScript, CSS) を開発の基盤として採用した。

1. はじめに

現代の音楽制作において、Digital Audio Workstation (DAW) は中心的な役割を担っている。市販の DAW は多機能かつ高性能であるが、一方で、必ずしも全てのユーザの特定のニーズやワークフローに合致しているとは限らない。機能が豊富になるほど動作が重く、低スペックな PC では快適な操作が困難な場合がある。さらに、ユーザが独自に機能を拡張したり、不要な機能を削除したりするカスタマイズの自由度は極めて低いのが現状である。

このような背景から、本研究では、ユーザが自身の制作スタイルに合わせて必要な機能を自ら取捨選択し構築できる、DIY (Do It Yourself) 型の DAW システム「DAWIY」を提案する [1]。DAWIY の目的は、柔軟な GUI、軽量の動作、そして何よりも高い拡張性を備えたクロスプラットフォームな音楽制作環境の実現である。以下、本稿では、DAWIY の設計思想、システムの基盤、および独自のプラグインシステムについて述べる。

2. 設計思想

冒頭で述べたように、DAWIY の最大の特徴は、ユーザが自身のニーズに合わせて機能を取捨選択し、理想的な音楽制作環境を DIY できる点にある。90 年代からの DAW の歴史を通じて、長期間にわたり発展してきた多くの DAW において、時代の変遷とともに独自の機能やメニュー、UI が

追加されてきたことにより、新規ユーザの参入や既存ユーザのソフトウェア移行、複数のソフトウェアで異なる機能の互換的な利用が困難になるケースが多々見受けられる。また、近年では AI 技術の発展に伴い、音楽制作における AI 支援機能への関心が高まっているが、既存の DAW においては、AI 支援機能の導入が限定的であり、ユーザが自らの制作スタイルに合わせて AI 機能を柔軟に追加・削除することが難しい状況にある。

これらの課題を解決するために、本研究では、以下の設計思想に基づいて、必要最低限の基本機能のみを備えたシンプルなコアシステムを提供し、ユーザが必要に応じて機能を追加・削除できる柔軟な拡張システムを提供する。これにより、ユーザは自分の制作スタイルに最適化された DAW を構築できる。

- **モジュール化された機能構成：**DAWIY は、基本機能と拡張機能を明確に分離し、ユーザが必要な機能のみを選択して利用できるように設計されている。これにより、ユーザは自身の制作スタイルに最適化された DAW を構築できる。
- **柔軟な UI カスタマイズ：**画面の色や配置、ショートカットキーなど、ユーザが自由にカスタマイズできる仕組みを提供している。これにより、ユーザは自身の好みや作業効率に合わせたインターフェースを実現できる。
- **拡張可能なプラグインシステム：**独自の DAWIY プラグインシステムを導入し、ユーザが自ら機能を追加・削除できるようにしている。さらに、AI アシスタントを活用したプラグイン生成機能を実装し、ユーザが自

¹ 福知山公立大学情報学部

^{a)} 32545100@fukuchiyama.ac.jp

^{b)} hashida-mitsuyo@fukuchiyama.ac.jp



図 1: DAWIY システム構成

然言語で要望を入力するだけで、独自の機能を迅速に開発・導入できるようにしている。

3. DAWIY のシステム構成

DAWIY^{*1}は、WAM-Studio[2]^{*2}を技術基盤とし、HTML, JavaScript, TypeScript, CSS を組み合わせて、web ブラウザおよびデスクトップ上で動作するクロスプラットフォームな DAW である。図 1 に DAWIY のシステム構成を示す。以下、本章では、DAWIY を構成する主要な機能と、そのカスタマイズ性について述べる。

3.1 基本制御機能

この項では、DAWIY にあらかじめ実装されている、DAW として必須の機能について述べる。図 2 に DAWIY の基本画面を示す。

- (1) オーディオ・MIDI 入出力の制御 PC に登録されているオーディオの入出力デバイス、MIDI の入出力デバイスをそれぞれ選択できる。
- (2) 再生制御と録音 再生／一時停止／停止ボタンによる演奏再生の制御を行う。また、録音ボタンを通じて、オーディオおよび MIDI データの記録を行う。
- (3) タイムラインとトラック管理 複数のトラックを追加・削除することができる。各トラックで音声や MIDI イベントをタイムラインに配置する。
- (4) ピアノロール トラックをダブルクリックすると、ピアノロール画面に遷移する(図 3)。ピアノロール画面では、マウス操作を通じて、(i) ノートの追加・削除、(ii) 再生位置の指定、(iii) ノートの長さの伸縮、(iv) 複数ノートの範囲選択、ほか、PC の標準的な編集ショートカットキーを操作できる。

3.2 表現制御機能

本項では、各トラックにおける表現力を高めるための機能について述べる。

- (1) オートメーションレーン 各トラック内の制御パラ

^{*1} <https://github.com/NumberZero-0621/DAWIY>

^{*2} Web Audio Modules (WAMs)[3] という、Web ベースの DAW において VST (Virtual Studio Technology) プラグインのように音源やエフェクト (FX) を追加するために作られた共通規格。

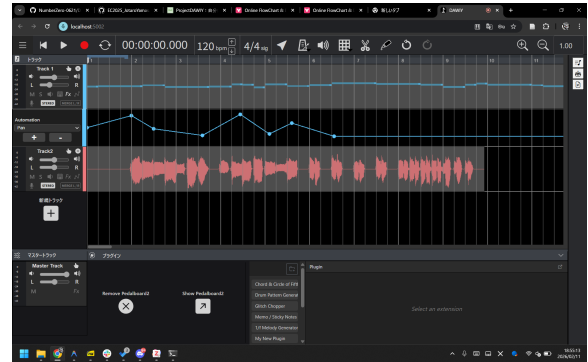


図 2: DAWIY の基本画面

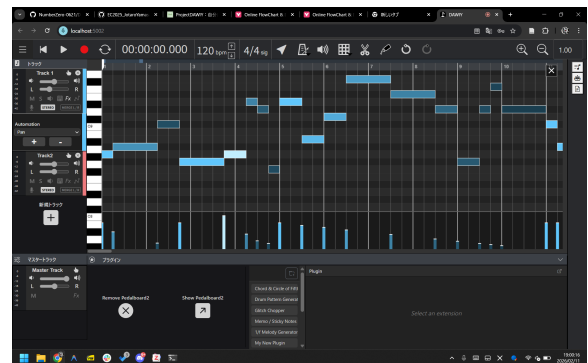


図 3: ピアノロール

メータ (音量、パン、エフェクトの各種パラメータなど) に対して、時間軸進行に伴いパラメータを動的に変化させることができる (図 2)。オートメーションレーンは編集対象とするパラメータの数に合わせて動的に増減できる。

- (2) ベロシティレーン ピアノロール画面の中に、MIDI ノートの強弱を制御するためのベロシティレーンを実装した (図 3)。ノートを追加・編集する際に、ノートのベロシティ値を個別に設定することができる。また、複数ノートを範囲選択した状態でベロシティレーン上でドラッグ操作を行うことで、選択中のノート全てのベロシティ値を線形に変化させることが可能である。

3.3 DIY 制御機能

- (1) プロジェクトファイルの相互入出力

他の DAW との互換性を確保するため、dawproject[4]形式のプロジェクトファイルを読み込み、DAW 内に展開するインポート機能を実装した。逆に、本 DAW 上で作成したシーケンス情報を .dawproject ファイルとして書き出すエクスポート機能も実装した。これにより、Studio One や Cubase といった他の主要 DAW とのプロジェクトデータの交換が可能となる。

- (2) 画面配置のカスタマイズ

画面左上のハンバーガーメニューから開く設定ウィンドウにて、ショートカットキーを自由に変更したり、メ

ニューの管理を行うことができる。また、トラックビューやピアノロール画面など、DAW を構成する各コンポーネントは、ドラッグアンドドロップで配置を自由に入れ替えたり、配色を変更したりすることができる。これらの機能を通じて、各ユーザが自身の好みや作業効率に合わせたインターフェースを実現できる。

(3) VST プラグイン

VST は、DAW 上で動作する、エフェクトや音源のプラグイン規格で、DAW コミュニティにおいて広く普及しているものである。ユーザの PC 内にある VST プラグインを検索・登録することで、DAWIY 内からも VST プラグインを呼び出し、楽器として音を鳴らしたりエフェクト処理をかけることが可能となる。

(4) DAWIY プラグイン

既存の DAW や VST プラグインの枠組みを超えて、より機能面に踏み込んだカスタマイズや、大幅な UI の変更・拡張を実現することを目的として、ユーザが独自に編集・表現機能を追加できる仕組みを提供する。詳細については次章で述べる。

4. DAWIY プラグイン

DAWIY プラグインは、VST プラグインではカバーできない、DAW の入出力・UI・操作等に介入する処理を拡張するための独自のプラグインシステムである。DAWIY プラグインを用いることで、それぞれのユーザにとって理想的な音楽制作環境を獲得することを目的としている。ユーザは自由に独自の DAWIY プラグインを作成することができ、簡単に機能を拡張することができる。図 4 に DAWIY プラグイン導入の流れを示す。

4.1 種類と適用範囲

DAWIY プラグインは、AviUtl 及び AviUtl2[5] のプラグインを参考にし、プラグインを下記のカテゴリに分類する。プラグインによっては、複数のカテゴリを横断する機能を保有することも可能である。また、下記のいずれかにプラグインとして実装できそうなものであっても、その機能が DAWIY ユーザ全体に必要である、と判断される場合には、プラグインとして配布するよりもプルリクエストを出すなどして DAWIY 本体に取り込むことを推奨する。

- **生成**：ノート、オーディオ、パターンを生成して楽曲に追加するプラグイン。
- **加工**：既存のノートやオーディオデータを選択・加工するプラグイン。
- **解析**：楽曲情報の可視化・分析を行うプラグイン。
- **入出力**：外部ファイル形式の読み込み・書き出し機能を追加するプラグイン。
- **UI**：DAW の全体的な外観の変更や、操作性の拡張を行うプラグイン。

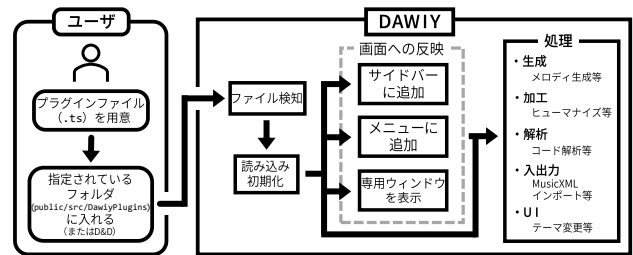


図 4: DAWIY プラグイン導入の流れ

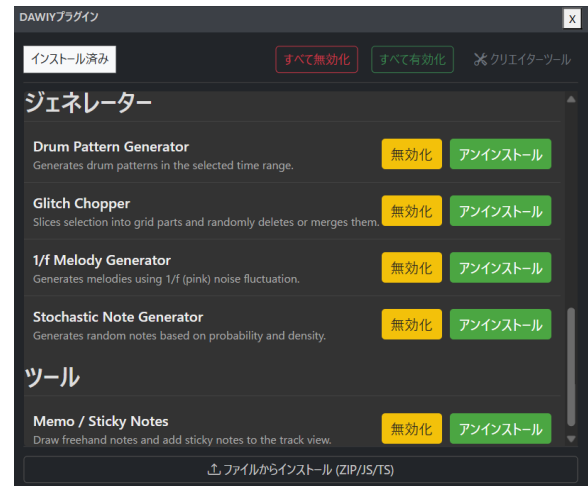


図 5: DAWIY パッケージマネージャ

これらのカテゴリは IDawiyPlugin インターフェース内で定義されており、プラグイン実装時に指定する。

4.2 全体管理

DAW 内部のパッケージマネージャ(図 5)より、登録されている DAWIY プラグインのインストール・アンインストール、また有効化・無効化が可能である。

エンドユーザは、パッケージマネージャからインストールボタンを押すか、事前にダウンロードした zip ファイル等を選択するか、DAW 画面上にドラッグアンドドロップするだけで自動的にプラグインをインストールすることができるため、内部のディレクトリ構造を把握する必要が無い。

4.3 作成方法

DAWIY プラグインは個々のユーザで自由に開発・導入することが可能である。DAWIY プラグインは、public/src/DawiyPlugins ディレクトリ以下に配置された TypeScript ファイル(.ts)として管理される。システムはこのディレクトリを自動的にスキャンし、有効なプラグインファイルを検出してロードする仕組みとなっている。プラグインごとの設定は、基本的には TypeScript のクラス定義内に記述されたメタデータ(ID, 名前, 説明など)が使用される。

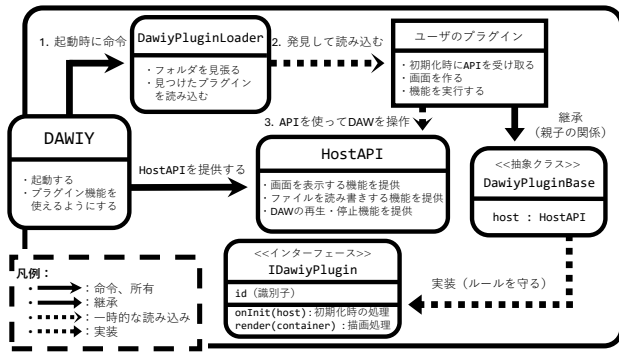


図 6: プラグイン周辺のクラス構造と連携



図 7: クリエイターツール

(1) クラス設計

図 6 に示すように、全てのプラグインは、基底クラスである `DawiyPluginBase` を継承し、`@DAWIYPlugin` デコレータを付与したクラスとして定義する必要がある。基本的な実装の流れは以下の通りである。

- (1) `DawiyPluginBase` クラスを継承したクラスを作成する。
- (2) クラスプロパティとして、一意な ID、名称、説明などを定義する。
- (3) 必要に応じて、初期化処理を行う `onInit` メソッド、UI を描画する `render` メソッド、有効化・無効化時の処理を行う `onActivate` / `onDeactivate` メソッドをオーバーライドして実装する。

プラグイン内からは、`HostAPI` を通じて DAW のコア機能 (UI 操作、ファイルシステムアクセス、トラックデータの操作など) に安全にアクセスすることが可能である。

ファイルを作成したら、DAW 内部のクリエイターツール (図 7) にファイルをドラッグアンドドロップし、名称、クラス名 (ID)、グループ名、説明等を記述してから生成ボタンを押すことで、プラグインを登録することが可能である。

(2) AI アシスタント

プラグイン開発を補助するツールとして、DAWIY の画面右側に常駐する「AI アシスタント」(サイドバー) を用意している (図 8)。

AI アシスタントは、Google Gemini API[6] や OpenAI

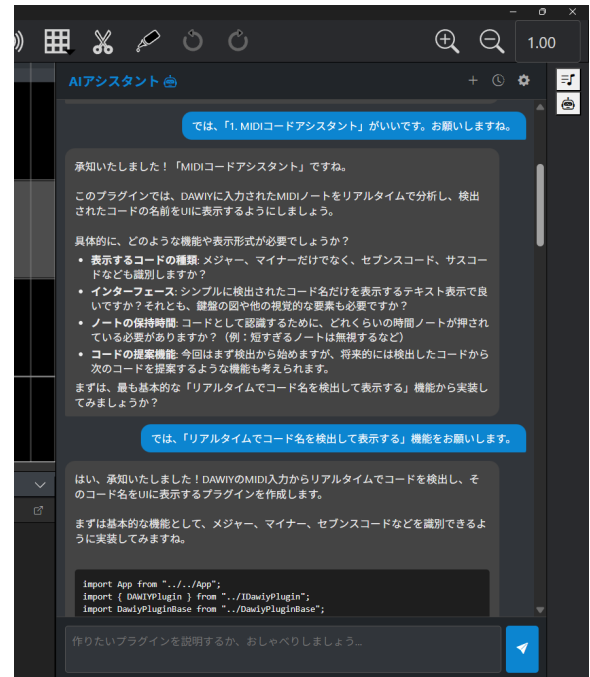


図 8: AI アシスタントのサイドバー

API[7] 等の LLM プロバイダと連携している。ユーザーが自然言語で「リアルタイムにコード名を検出して表示する機能を作成して」「ランダムにメロディを生成する機能が欲しい」といった要望を入力すると、システムはプロンプトを LLM へ送信し、要件定義から TypeScript コードの生成までを自動的に行う。生成されたコードは DAWIY プラグインとして即座にビルド・ロードされる。この一連の流れにより、ユーザーはコードを一切書くことなく、対話をするだけで数十秒で独自の機能を DAWIY に追加することが可能である。

4.4 事例

本研究では、DAWIY プラグインの有用性と表現力を検証するために、いくつかのプロトタイププラグインを実装している。

(1) MIDI Chord Assistant

リアルタイムに MIDI 入力信号を解析し、現在演奏されているコード (和音) の名前を表示する解析プラグインである。ユーザーが鍵盤などで和音を弾くと、その構成音から「C Major」や「A Minor 7」といったコード名を判定し、専用のウィンドウに表示する。これにより、演奏中や作曲中の音楽理論的な補助を行うことができる。

(2) Memo / Sticky Notes

トラックビュー上に、手書きのメモや付箋 (Sticky Note)、テキストボックスを自由に配置できる UI プラグインである。PIXI.js を利用したオーバーレイ描画により、楽曲の特定の箇所に対してアイデアや修正指示などを視覚的に残すことができる。通常の DAW のマーカー機能よりも自由度

が高く、視覚的な情報整理に役立つ。

(3) Stochastic Note Generator

確率的なアルゴリズムを用いて、ランダムなメロディやリズムパターンを生成する生成プラグインである。指定したスケールや確率分布に基づいてノートを生成し、トラックに配置する。作曲のアイデア出しや、偶然性を利用した音楽制作を支援する。

(4) Sample IO & UI Plugin

独自のファイル形式（.sampletext）の入出力を扱う機能と、サイドバーへのカスタム UI 追加を実証するためのプラグインである。テキストデータのインポート・エクスポート機能や、サイドバー内での独自コントロールの配置など、DAWIY の拡張性を網羅的に利用した実装例となっている。

5. 開発の現状と今後の展望

現在、DAWIY の基本機能となるピアノロール、オーディオ操作、およびプラグインシステムのコア部分は実装が完了しており、Web ブラウザ上で動作する DAW として最低限の機能を有している。特に、AI アシスタントを用いたプラグイン生成フローは、従来の DAW にはない革新的な機能として、ユーザによるカスタマイズの敷居を大きく下げること成功している。しかし、3 章で述べた機能のうち、カラーテーマの指定、ドラッグアンドドロップによる GUI の配置の移動などについては執筆時点で開発段階にある。

今後の展望として、(1) HostAPI の拡充を通じて、より高度な DAW 操作をプラグインから可能にすること、(2) AI アシスタントの精度向上によるプラグイン生成の信頼性向上、および (3) DAWIY プラグインをユーザ間で共有する仕組みの構築を挙げる。これらの改善により、DAWIY のコミュニティを活性化させ、多様な機能が自律的に生まれる環境を作ることが期待される。

謝辞 本研究を行うにあたって、プラグインの開発等に関わってくださった平山 心^{*3}氏に深く感謝申し上げる。

参考文献

- [1] 承 太 郎 山 崎 : NumberZero-0621/DAWIY, <https://github.com/NumberZero-0621/DAWIY>.
- [2] Buffa, M.: WAM-studio, a Digital Audio Workstation (DAW) for the Web | Companion Proceedings of the ACM Web Conference 2023, <https://dl.acm.org/doi/abs/10.1145/3543873.3587987>.
- [3] Buffa, M., Ren, S., Campbell, O., Burns, T., Yi, S., Kleimola, J. and Larkin, O.: Web Audio Modules 2.0: An Open Web Audio Plugin Standard, *Companion Proceedings of the Web Conference 2022, WWW '22*, New York, NY, USA, Association for Computing Machinery, pp. 364–369 (online), DOI: 10.1145/3487553.3524225 (2022). <https://dl.acm.org/doi/10.1145/3487553.3524225>.
- [4] : Bitwig/Dawproject, Bitwig (2025).

<https://github.com/bitwig/dawproject>.

- [5] K E N くん : AviUtl の お 部 屋 , <https://spring-fragrance.mints.ne.jp/aviutl/>.
- [6] : Gemini API, <https://ai.google.dev/gemini-api/docs?hl=ja>.
- [7] : OpneAI API, <https://openai.com/ja-JP/api/>.

^{*3} 関西学院大学工学部, ifp34362@kwansei.ac.jp